

Plexys Homework

Support Ticket Management - Corteza + Vue 3

Corteza version	2024.9.9 - latest stable release at time of writing
Frontend stack	Vue 3 (Composition API) + PrimeVue
Serving method	Standalone build, independent of Corteza's bundled web-app hosting

1. Architecture

The solution has three independently deployed services, all containerized via Docker Compose:

```

Browser (Vue 3 SPA)
├── Corteza REST API (tickets, customers, records)
├── Go auth-proxy (OAuth2 token exchange only)
└── Corteza OAuth2 server (internal Docker network)

```

Corteza runs via Docker Compose with PostgreSQL 15. An init container automates admin account creation on first boot, mirroring Corteza's normal sign-up flow.

Why a separate Go auth-proxy exists

Corteza's authorization_code OAuth2 flow expects a confidential client that authenticates itself with a secret during token exchange. A single-page app cannot keep that secret hidden - anything referenced in frontend code ships as plain-text JavaScript to every visitor; embedding it via an env file does not change this.

PKCE (which removes the need for a secret entirely) was investigated as the standards-based fix for public clients, but Corteza's Auth Client admin UI exposes no explicit public-client/PKCE toggle, and server-side support could not be confirmed without risking a broken submission. Given an existing Go/Gin microservice was already available, extending it with a two-endpoint token-exchange proxy was the correct, minimal fix: the secret lives only in the proxy's server-side configuration and never reaches the browser.

2. Design Decisions

Vue 3, Composition API, throughout

All components use `<script setup>` with `ref()/computed()/onMounted()` - no Options API, no mixins.

Values as name/value pairs, not flat objects

Corteza's Compose record API represents field values as an array of `{name, value}` pairs on both read and write. A normalization step flattens this on fetch so templates use simple property access; a corresponding builder converts back on save.

Desktop table / mobile cards via CSS breakpoints, not a UI library's responsive mode

PrimeVue's DataTable ships a built-in responsive-stacking mode that auto-injects column labels into collapsed rows, which produced layout bugs difficult to control since that DOM structure is internal to the library. Replaced with two independently-styled templates - a real table on desktop, hand-written cards on mobile - toggled via CSS breakpoints.

Reactive auth state via a shared ref, not raw localStorage reads

An early version gated navigation on a `computed()` reading `localStorage` directly, which is not reactive in Vue. Fixed by exporting a shared ref, flipped explicitly inside login/logout.

Session-expiry handling

Corteza access tokens expire after 2 hours; an expired token can surface as a 500 rather than a clean 401. A response interceptor treats 401/403/500 uniformly as an auth failure and redirects to login - a deliberate, time-boxed trade-off (see Known Limitations).

Dedicated Auth Client, not the platform default

The default "Corteza Webapps" client was confirmed to work, but a separate client (plexys) was created deliberately instead of reusing it - each integration should have its own client so scope, secret rotation, and revocation can be managed independently without risking Corteza's own bundled UI.

Design patterns applied

Proxy - the backend service exists specifically to sit between the frontend and Corteza, forwarding the token exchange while adding something the browser cannot be trusted with (the client secret).

Interceptor - axios request/response interceptors centralise bearer-token injection and session-expiry handling so individual API calls stay unaware of auth.

Adapter - normalizeValues()/buildPayloadValues() convert between Corteza's {name, value} pair array and the flat objects the UI works with, isolating the rest of the app from that API shape.

Layered architecture and constructor-based dependency injection in the Go service (handlers → services → config), each layer unaware of the one above it.

Idempotent provisioning - dev-seed.mjs checks namespace/module/field existence before creating anything, safe to re-run without side effects.

Singleton - the exported isAuthenticated ref is created once and shared everywhere it is imported, giving the whole app one source of truth for auth state rather than per-component copies.

Guard - router.beforeEach checks route meta before rendering, redirecting unauthenticated users away from protected views.

Observer - Vue's own reactivity: the nav re-renders the instant isAuthenticated flips, with no manual DOM update code.

Middleware chain - Gin's CORS handling and the rate limiter both wrap the request pipeline, the limiter scoped only to the token-exchange route rather than applied globally.

Token bucket - the rate limiter's actual algorithm (golang.org/x/time/rate), smoothing bursty requests to the unauthenticated exchange endpoint rather than a hard cutoff.

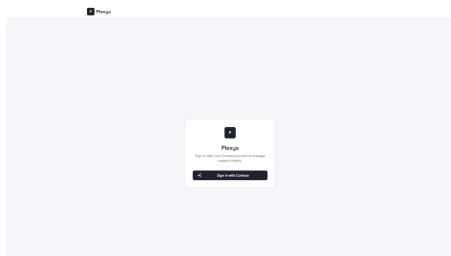
Init/bootstrap - corteza-init runs once to create the administrator account, then exits, distinct from the long-running services around it.

Backend-for-frontend - the auth-proxy is narrowly scoped to this one frontend's specific need (safe token exchange), not a general-purpose API.

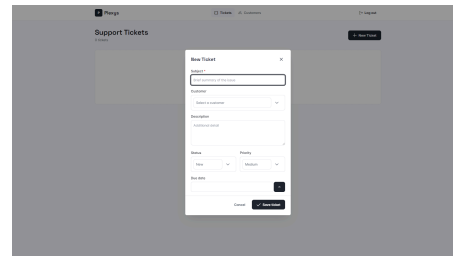
Module pattern - auth.js and cortezaClient.js each encapsulate private state and expose a controlled public API rather than exposing raw variables.

3. User Guide

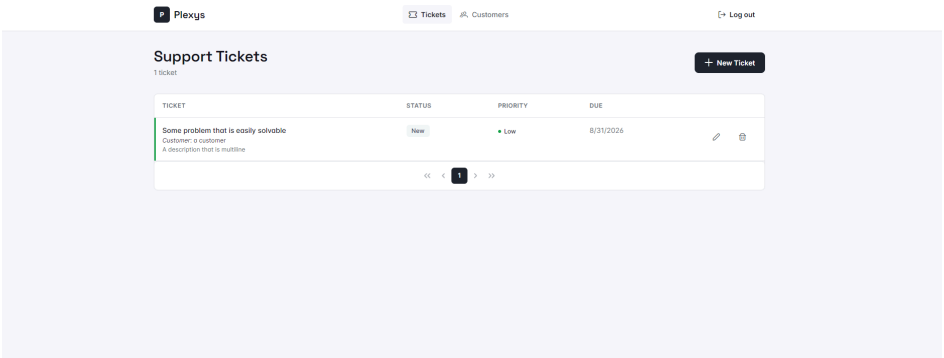
1. Sign in via Sign In with Corteza - redirects to Corteza's own login page.
2. The Tickets view lists all support tickets; a coloured bar on the left of each row indicates priority.
3. New Ticket: complete Subject (required), Customer, Description, Status, Priority and Due Date, then Save.
4. Edit via the pencil icon; delete via the trash icon (immediate, no confirmation step).
5. Customers (header nav) manages the customer list the same way.
6. On narrow screens the nav becomes icon-only and the ticket list becomes a stack of cards.



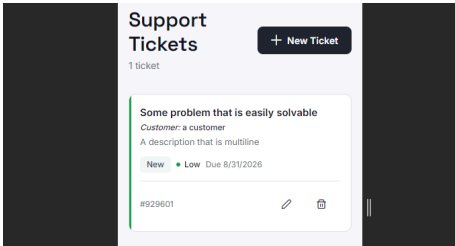
Sign-in screen



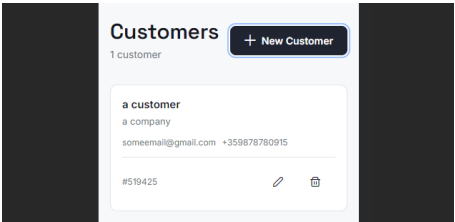
New Ticket dialog



Ticket list - desktop, with priority indicator and customer



Mobile card view



Customers - mobile

4. Admin Guide - clean-clone environment setup procedure

Support Ticket Management System - Corteza + Vue 3

Document type	Clean-clone environment setup procedure
Audience	Developers and reviewers standing up the project for the first time
Prerequisites	Docker Desktop; Docker Compose V2; Node.js 18+

4.1. Overview

This part describes the complete procedure for taking a fresh clone of the repository to a fully running, authenticated application. The stack comprises four Docker-managed services - a PostgreSQL database, the Corteza server, a purpose-built Go authentication proxy, and the Vue 3 frontend - plus one manual configuration step performed through the Corteza administration interface, and one setup script run once against the freshly configured instance.

Two categories of configuration exist in this project and should not be confused: values that are safe to expose to the browser (prefixed VITE_, living in frontend/.env) and the OAuth2 client secret, which must never leave the backend service's own configuration file. This distinction is called out explicitly at the relevant step below.

4.2. Clone and Prepare Environment Files

```
git clone https://github.com/yazu-codes/plexys-task.git
cd plexys-task
```

Create working copies of every environment template. None of the real configuration files are committed to source control.

```
cp .env.example .env
cp frontend/.env.example frontend/.env
cp backend/configs/config_example.yaml backend/configs/config.yaml
```

Open the root .env file and set the administrator credentials that will be created automatically on first boot (they are populated by default with admin@plexys.eu and SecurePassword123!):

```
ADMIN_EMAIL=your-admin-email
ADMIN_PASSWORD=your-strong-password
```

4.3. Start the Stack

Bring up every service defined in the offline Docker Compose configuration:

```
docker-compose -f docker-compose.offline.yml up -d
```

The corteza-init service runs once, creates the administrator account from the credentials set in the previous step, and then exits - this is expected behaviour, not a failure. Confirm the stack is healthy:

```
docker-compose -f docker-compose.offline.yml ps
```

Confirm the Corteza API is reachable:

```
curl http://localhost:18080/version
```

4.4. Create the Authentication Client

The frontend authenticates against Corteza using the OAuth2 authorization_code grant. This requires a dedicated Auth Client, created once through the Corteza administration interface.

1. Open <http://localhost:18080> and sign in using the administrator credentials configured in Section 2.
2. Navigate to Admin → System → Auth Clients, and select New Auth Client.
3. Enter Name and Handle: plexys.
4. Under Redirect URI's, add: <http://localhost:5173/callback>.
5. Select "Will be used to authenticate users (grant type = authorization_code)".
6. Under permissions, check "Allow client access to user's profile" and "Allow client access to Corteza API on behalf of user". Leave OIDC and Discovery API unchecked.
7. Ensure Enabled and Trusted are both checked.
8. Select 'Default user group for new users' to be 'Default Sub-User Group'.
9. Press Submit.

- Reopen the client and note the Client ID shown in the browser's address bar, and the Client Secret shown on the client's detail page.

localhost:18080/admin/system/authClient/edit/510296434915344385

New Auth Client Permissions

Basic information

Name * plexys **Handle** plexys

Redirect URI's
 ✕
 + Add

Secret

EZlxZKG3F0z5LyemuPWrr2KALWmpdeFRWkQBUAei8F574RSqthJ4vaqcgG9AU

☒ Will be used to authenticate users (grant type = authorization_code)
☐ Will be used to authenticate machines (grant type = client_credentials)

Valid from
☐ None ☐ None
If not defined the client is valid until expiration

☒ Enabled

Permitted roles
 ▼

Forced roles

☒ Allow client access to user's profile
☒ Allow client access to Corteza API on behalf of user
☐ Allow client to use OIDC and verify the user's identity
☐ Allow client access to Corteza Discovery API on behalf of user

Expires at
☐ None ☐ None
If not defined the client has no expiration date

☒ Trusted
When client is trusted users do not see authorization step

Prohibited roles
 ▼

Note: I tried using the default auth client as well and it worked, but figured it is not the best practice to NOT create a new auth client for the task.

4.5. Apply Client Credentials

Security note

The Client Secret must be written only to backend/configs/config.yaml. It must never appear in frontend/.env or any file prefixed VITE_, since such values are compiled into client-side JavaScript and are visible to anyone who inspects the application in a browser.

In frontend/.env, set:

```
VITE_OAUTH_CLIENT_ID=<Client ID from Section 4> ← IMPORTANT
VITE_OAUTH_REDIRECT_URI=http://localhost:5173/callback
VITE_AUTH_PROXY_URL=http://localhost:8080
```

In backend/configs/config.yaml, under the corteza: section, set:

```
corteza:
  client_id: "<Client ID from Section 4>" ← IMPORTANT
  client_secret: "<Client Secret from Section 4>" ← IMPORTANT
  auth_base_url: "http://corteza-server/auth"
  redirect_uri: "http://localhost:5173/callback"
```

Docker networking note

auth_base_url uses the Docker Compose service name corteza-server rather than localhost. The authentication proxy runs inside a container and must reach Corteza over the internal Docker network; localhost from within a container refers to that container itself, not the host machine.

4.5.1 Restart frontend and backend services

```
docker-compose -f docker-compose.offline.yml restart backend frontend
```

4.6. Populate the Workspace (a quick manual for namespace and modules population)

Go to localhost:5173 (frontend's local address and port) and click the login button. **Open the browser's developer tools, navigate to Application → Local Storage**, and copy the value of the key **holding the session JWT**.

Run the provisioning script with this token:

```
node scripts/dev-seed.mjs <token>
```

This script creates the "Plexys Homework" namespace together with the Support Ticket and Customer modules and all associated fields, if they do not already exist. It is idempotent and safe to run repeatedly - existing namespaces, modules, and fields are detected and left untouched. On completion it prints three identifiers.

Copy these into frontend/.env:

```
VITE_CORTEZA_NAMESPACE_ID=<namespace ID>
VITE_CORTEZA_TICKET_MODULE_ID=<ticket module ID>
VITE_CORTEZA_CUSTOMER_MODULE_ID=<customer module ID>
```

4.7. Apply Configuration

The backend and frontend services were started in Section 3 before this configuration existed. Restart both so they pick up the values written in Sections 5 and 6:

```
docker-compose -f docker-compose.offline.yml restart backend frontend
```

4.8. Verification

1. Open <http://localhost:5173>.
2. Press Sign In with Corteza. The browser will redirect to Corteza's own login page.
3. Sign in with the administrator credentials.
4. Confirm the browser returns to the application and displays the Tickets view.
5. Create, edit, and delete a ticket. Repeat for a customer record, and confirm the customer can be associated with a ticket.

4.9. Troubleshooting

Inspect logs for the full stack, or an individual service:

```
docker-compose -f docker-compose.offline.yml logs -f
```

```
docker-compose -f docker-compose.offline.yml logs corteza-server
docker-compose -f docker-compose.offline.yml logs backend
docker-compose -f docker-compose.offline.yml logs frontend
```

Corteza reports unhealthy after a host disk-space issue or unclean shutdown

Remove the container and its image, then allow Docker Compose to pull a fresh copy, rather than assuming the underlying data volume is corrupted: `docker-compose stop corteza-server`, `docker-compose rm -f corteza-server`, `docker rmi cortezaproject/corteza:2024.9`, then `docker-compose up -d corteza-server`.

4.10. Full Reset

The following removes all containers, volumes, and images associated with this project, including the Corteza database. The Auth Client, namespace, and modules configured in Sections 4 through 6 will need to be recreated.

```
docker-compose -f docker-compose.offline.yml down -v --rm all
docker-compose -f docker-compose.offline.yml build --no-cache
```

Then repeat the procedure from Section 3 onward.

5. Known Limitations & Next Steps

- Delete has no confirmation dialog - accepted given time budget; an obvious next addition.
- Session-expiry handling is broad (401/403/500 treated uniformly) rather than distinguishing genuine auth failures from unrelated server errors.
- Client-side pagination only - all records fetched in one request and paginated in the browser; fine at this scale, would need Corteza's `page/perPage` params for a larger dataset.
- No refresh-token flow - the refresh token is stored but unused; expiry is handled reactively rather than proactively renewed.
- Customer relationship display is a client-side lookup against a fetched list, not a server-side join - acceptable at this scale.

Production deployment

docker-compose.online.yml and the **production Dockerfile** exist and mirror the offline setup's topology (Nginx, Let's Encrypt, production build), but have not been deployed or tested - only the offline path has been verified end to end. Before real use, this would need: working TLS/domain routing and cert issuance, the OAuth2 client and CORS updated to real domains, the auth-proxy's secret moved to environment variables instead of a mounted file, and a real load test of the rate limiter and session handling. A VPS staging deploy using the existing compose file would be the fastest way to surface any gaps.

CORS configuration

The **auth-proxy** currently allows all origins (`AllowAllOrigins: true`), left over from development rather than restricted to the known frontend. Combined with credentialed requests, this is broader than necessary and should be tightened to a fixed allow-list before any exposure beyond local development. Identified but left as-is here - correct for its current audience (one local frontend), and deprioritised in favour of finishing required functionality within the time budget.